

Tutorial: Middlewares en NestJS (log + tiempo de respuesta)

Pequeño recorrido para agregar middlewares HTTP a tu API Nest. Solo vas a hacer dos cosas: **ver cada petición en consola** y, después, **agregar una cabecera que mide tiempo**.

1. ¿Qué es esto?

Cuando llega una petición (por ejemplo `GET /products`), Nest la pasa por varias capas. Un **middleware** es código que corre **antes** de llegar al controlador (`@Controller`).

Sirve sobre todo para logs, métricas sencillas o armar objetos/`req` comunes.

En esta guía vas a crear **clases `NestMiddleware`**, igual que muestra la documentación oficial de Nest (sección **Middleware**).

Pasos dentro de cada middleware:

1. Hacés lo que necesites (log, cronómetro, etc.).
 2. **Siempre llamás `next()`** para que siga la cadena hasta el controller. Sin `next()` la petición se queda colgada.
-

2. Parte primera — Middleware de log

Consigna (simple): cada vez que llegue una petición, mostrar en **la consola del servidor** cosas útiles como el **método** (GET, POST), la **URL** y la **hora** (podés usar `new Date().toISOString()`).

2.1 Crear archivo y clase

Carpeta recomendada: `src/common/middlewares/` (si no existe, creala).

Creá ahí `logger.middleware.ts` con algo así (completando el log):

```
import { Injectable, NestMiddleware } from '@nestjs/common';
import type { NextFunction, Request, Response } from 'express';

@Injectable()
export class LoggerMiddleware implements NestMiddleware {
  use(req: Request, res: Response, next: NextFunction) {
    // Acá tu console.log: método, url (req.originalUrl o req.url) y fecha/hora
    next(); // ¡no lo olvides!
  }
}
```

Recordatorio: Nest usa Express por debajo (`@nestjs/platform-express`), por eso los tipos `Request/Response` vienen de **express** y están tipados así.

Ejecutá `npm run build` para comprobar que no hay errores de TypeScript antes de seguir.

2.2 Enchufar el middleware al módulo raíz

Nest no “adivina” el middleware; hay que decir **dónde** corre. Lo habitual es el AppModule.

1. Abrió `src/app.module.ts`.
2. Hací que **AppModule implemente la interfaz NestModule**.
3. Agregá el método `configure` con **MiddlewareConsumer**.

Ejemplo típico (ajustando imports):

```
import { MiddlewareConsumer, Module, NestModule } from '@nestjs/common';
// imports de tu proyecto: TypeOrm, ConfigModule, otros módulos...
import { LoggerMiddleware } from '../common/middlewares/logger.middleware'; // rutas
según dónde lo creaste

@Module({
  // imports, controllers y providers igual que ya tenías
})
export class AppModule implements NestModule {
  configure(consumer: MiddlewareConsumer): void {
    consumer.apply(LoggerMiddleware).forRoutes('*');
  }
}
```

Qué hace cada pieza:

- **MiddlewareConsumer** es el objeto que Nest te inyecta en `configure` para registrar middlewares.
- **`.apply(LoggerMiddleware)`** dice cuál clase usás.
- **`.forRoutes('*')`** dice “todas las rutas”. Más abajo ves cómo achicarlo.

Probá de nuevo con `npm run build`, luego `npm run start:dev`. Desde el navegador o con `curl http://localhost:3000/` y otra ruta (por ejemplo `/products`) deberías ver **nuevas líneas en la terminal** del servidor.

2.3 (Opcional) Que el log solo corra en una parte de la API

Si cambiás `forRoutes('*')` por `forRoutes('products')`, el logger corre solo bajo rutas que empiezan con `/products`. También podés usar `forRoutes(ProductsController)` importando el controller.

Es útil cuando un middleware es caro o no aplica a todo (por ejemplo solo a `/admin`).

3. Parte segunda — Cabecera X-Response-Time

Consigna (simple): además del log, que la respuesta lleve un header **X-Response-Time** con cuántos **milisegundos** tardó la petición en el servidor (desde que entra al middleware hasta que Express termina de armar la respuesta que manda con `send`).

3.1 Por qué no usamos solo el evento `finish`

El evento **`finish`** de la respuesta se dispara **cuando ya se envió todo**. En ese momento **no se pueden agregar headers**. Por eso acá usamos un truco clásico de Express: guardamos el `send`

original y **antes** de llamarlo agregamos el header.

3.2 Nuevo archivo `timing.middleware.ts`

Creá en la misma carpeta que el logger un middleware parecido a este (podés ajustar nombres y tipos):

```
import { Injectable, NestMiddleware } from '@nestjs/common';
import type { NextFunction, Request, Response } from 'express';

@Injectable()
export class TimingMiddleware implements NestMiddleware {
  use(req: Request, res: Response, next: NextFunction): void {
    const startAt = Date.now();
    const originalSend = res.send.bind(res);

    res.send = (body: unknown) => {
      const ms = Date.now() - startAt;
      res.setHeader('X-Response-Time', `${ms} ms`);
      return originalSend(body);
    };

    next();
  }
}
```

Si querés **ver el mismo número en consola**, agregá un `console.log` ahí mismo con `ms`.

3.3 Registrar los dos middlewares

En `configure` podés encadenar varios. **El orden en `apply` es el orden en que se ejecutan** al entrar la petición:

```
configure(consumer: MiddlewareConsumer): void {
  consumer.apply(LoggerMiddleware, TimingMiddleware).forRoutes('*');
}
```

Primero corre el logger, luego el de tiempo (ambos llaman a `next()` y el siguiente recibe el control).

3.4 Probar el header

Con el servidor levantado:

```
curl -i http://localhost:3000/products
```

En la salida deberías ver una línea parecida a `X-Response-Time: 12 ms` (el número varía).

4. Checklist rápido (para vos)

- ☐ Log con método, URL y hora en consola.
- ☐ `AppModule` implementa `NestModule` y tiene `configure` con `MiddlewareConsumer`.

- ☐ Header `X-Response-Time` visible con `curl -i` en alguna ruta.
-

5. Para pensar (opcional)

- ¿Qué hace `next()` y qué pasaría si no lo llamás?
 - ¿En qué se parece y en qué se diferencia un **middleware** de un **interceptor** de Nest?
-

Guía de laboratorio — middlewares HTTP en NestJS.